In [1]:

```
# MantisV2 improvement: learned token pooling (idea 2) + cross-channel mi
xer (idea 1) on the frozen encoder
# Padmanabh Khunt (26100085), AI for Time Series group project
!pip install -q mantis-tsfm
import os, time, json, zipfile, subprocess, requests
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from mantis.architecture import MantisV2
from mantis.trainer import MantisTrainer

device = 'cuda'
DATA, OUT = '/tmp/data', '/kaggle/working'
os.makedirs(DATA, exist_ok=True)
net = MantisV2(device=device, return_transf_layer=2, output_token='combin
ed').from_pretrained('paris-noah/MantisV2')
net.return_transf_layer, net.output_token = 2, 'combined'
net.eval()
print(torch.cuda.get_device_name(0))
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Pleas
e set a HF_TOKEN to enable higher rate limits and faster downloads.
```

```
Tesla T4
```

In [2]:

```
# Data loaders (same as the reproduction notebook)
def load_ts_archive(name):
    folder = f'{DATA}/{name}'
    if not os.path.exists(folder):
        subprocess.run(['wget', '-q', '-O', f'{DATA}/{name}.zip',
                        f'https://www.timeseriesclassification.com/aeon-t
oolkit/{name}.zip'], check=True)
        with zipfile.ZipFile(f'{DATA}/{name}.zip') as z:
            z.extractall(folder)
    out = {}
    for split in ['TRAIN', 'TEST']:
        path = [os.path.join(r, f) for r, _, fs in os.walk(folder) for f
in fs if f == f'{name}_{split}.ts'][0]
        X, y, in_data = [], [], False
        with open(path, encoding='utf-8', errors='ignore') as fh:
            for line in fh:
                line = line.strip()
                if not line or line.startswith('#'):
                    continue
                if line.lower().startswith('@data'):
                    in_data = True
                    continue
                if in_data:
                    *channels, label = line.split(':')
                    X.append([[float(v) if v != '?' else np.nan for v in
ch.split(',')] for ch in channels])
                    y.append(label)
        out[split.lower()] = (np.asarray(X, dtype=np.float32), np.asarray
(y))
    return out

TFC = {'HAR': 19930244, 'Epilepsy': 19930199}

def load_tfc(name):
    folder = f'{DATA}/{name}'
    os.makedirs(folder, exist_ok=True)
    files = requests.get(f'https://api.figshare.com/v2/articles/{TFC[nam
e]}', timeout=30).json()['files']
    out = {}
    for split in ['train', 'test']:
```

```
        path = f'{folder}/{split}.pt'
        if not os.path.exists(path):
            url = [f['download_url'] for f in files if f['name'] == f'{sp
lit}.pt'][0]
            with open(path, 'wb') as fh:
                fh.write(requests.get(url, timeout=600).content)
        d = torch.load(path, weights_only=False)
        X, y = np.asarray(d['samples'], dtype=np.float32), np.asarray(d
['labels'])
        if X.ndim == 2:
            X = X[:, None, :]
        out[split] = (X, y)
    return out
```

In [3]:

```python
# Frozen encoder -> all tokens of all 6 transformer layers
@torch.no_grad()
def tokens(X, bs=512):
    """Returns (N, C, 6 layers, 33 tokens, 256) in float16. Tokens 0-31 a
re patches, token 32 is the cls token."""
    X = np.nan_to_num(np.asarray(X, dtype=np.float32))
    N, C, _ = X.shape
    X = F.interpolate(torch.tensor(X), size=512, mode='linear', align_cor
ners=False).reshape(N * C, 1, 512)
    out = []
    for i in range(0, N * C, bs):
        h = net.tokgen_unit(X[i:i + bs].to(device))
        cls = net.transf_unit.cls_token.expand(h.shape[0], 1, -1)
        h = torch.cat([h, cls], 1)[:, None]
        layers = []
        for layer in net.transf_unit.transformer.layers:
            h = layer(h)
            layers.append(h[:, 0])
        out.append(torch.stack(layers, 1).half().cpu())
    return torch.cat(out).reshape(N, C, 6, 33, 256)

def fixed_features(T):
    """The paper's features: layer 2, cls token + mean of patch tokens, c
hannels concatenated."""
    t = T[:, :, 2].float()
    return torch.cat([t[:, :, 32], t[:, :, :32].mean(2)], -1).reshape(len
(T), -1).numpy()

# sanity check: our token extraction must give the official MantisV2 feat
ures
Xc = np.random.randn(8, 2, 300).astype(np.float32)
official = MantisTrainer(device=device, network=net).transform(
    F.interpolate(torch.tensor(Xc), size=512, mode='linear', align_corner
s=False).numpy())
print('max difference to official features:', np.abs(fixed_features(token
s(Xc)) - official).max())
```

```
max difference to official features: 0.0037460327
```

In [4]:

```
# The new head on top of the frozen encoder
class Head(nn.Module):
    """pool='fixed': paper aggregation (layer 2, cls + mean token). pool
='learned': softmax-weighted mix of the
    6 layers + attention pooling over the 33 tokens (idea 2).
    chan='concat': channels concatenated as in the paper. chan='mixer': t
ransformer across channels (idea 1)."""
    def __init__(self, C, K, pool, chan, d=128, heads=4, drop=0.2):
        super().__init__()
        self.pool, self.chan = pool, chan
        if pool == 'fixed':
            self.proj = nn.Sequential(nn.LayerNorm(512), nn.Linear(512,
d))
        else:
            self.layer_w = nn.Parameter(torch.zeros(6))
            self.norm = nn.LayerNorm(256)
            self.query = nn.Parameter(torch.randn(heads, 256) * 0.02)
            self.proj = nn.Sequential(nn.LayerNorm(heads * 256), nn.Linea
r(heads * 256, d))
        if chan == 'concat':
            self.out = nn.Sequential(nn.Dropout(drop), nn.Linear(C * d,
K))
        else:
            self.chan_emb = nn.Parameter(torch.randn(1, C, d) * 0.02)
            self.cls = nn.Parameter(torch.zeros(1, 1, d))
            layer = nn.TransformerEncoderLayer(d, 4, 2 * d, drop, batch_f
irst=True, norm_first=True)
            self.mixer = nn.TransformerEncoder(layer, 1, enable_nested_te
nsor=False)
            self.out = nn.Sequential(nn.LayerNorm(d), nn.Dropout(drop), n
n.Linear(d, K))

    def forward(self, T):
        T = T.float()
        if self.pool == 'fixed':
            t = T[:, :, 2]
            z = torch.cat([t[:, :, 32], t[:, :, :32].mean(2)], -1)
        else:
            w = torch.softmax(self.layer_w, 0)
            t = (self.norm(T) * w[:, None, None]).sum(2)      # weighted
```

```
mix of the 6 layers: (B, C, 33, 256)
            a = torch.softmax(t @ self.query.T / 16, 2)       # attention
over the 33 tokens: (B, C, 33, heads)
            z = (a.transpose(2, 3) @ t).flatten(2)            # (B, C, he
ads * 256)
        z = self.proj(z)                                      # (B, C, d)
        if self.chan == 'concat':
            return self.out(z.flatten(1))
        z = torch.cat([self.cls.expand(len(z), -1, -1), z + self.chan_em
b], 1)
        return self.out(self.mixer(z)[:, 0])
```

In [5]:

```
# Training (20% of the train split held out for early stopping) and the l
ogistic regression baseline
def train_head(Ttr, ytr, Tte, yte, C, K, pool, chan, seed, epochs=100, pa
tience=20, bs=64):
    torch.manual_seed(seed)
    np.random.seed(seed)
    idx_tr, idx_va = train_test_split(np.arange(len(ytr)), test_size=0.2,
stratify=ytr, random_state=seed)
    model = Head(C, K, pool, chan).to(device)
    opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.0
5)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, epochs)
    ytr_t = torch.tensor(ytr, dtype=torch.long)

    def predict(T):
        model.eval()
        preds = []
        with torch.no_grad():
            for i in range(0, len(T), 512):
                preds.append(model(T[i:i + 512].to(device)).argmax(1).cpu
())
        return torch.cat(preds).numpy()

    best, best_state, wait = -1, None, 0
    for ep in range(epochs):
        model.train()
        perm = np.random.permutation(idx_tr)
        for i in range(0, len(perm), bs):
            b = torch.tensor(perm[i:i + bs])
            loss = F.cross_entropy(model(Ttr[b].to(device)), ytr_t[b].to
(device), label_smoothing=0.1)
            opt.zero_grad()
            loss.backward()
            opt.step()
        sched.step()
        va = np.mean(predict(Ttr[torch.tensor(idx_va)]) == ytr[idx_va])
        if va >= best:
            best, wait = va, 0
            best_state = {k: v.clone() for k, v in model.state_dict().ite
ms()}
```

```
        else:
            wait += 1
            if wait >= patience:
                break
    model.load_state_dict(best_state)
    n_params = sum(p.numel() for p in model.parameters())
    return float(np.mean(predict(Tte) == yte)), ep + 1, n_params

def logreg(Ztr, ytr, Zte, yte):
    s = StandardScaler().fit(Ztr)
    clf = LogisticRegression(max_iter=500).fit(s.transform(Ztr), ytr)
    return float(np.mean(clf.predict(s.transform(Zte)) == yte))
```

In [6]:

```
# Run: baselines (RF from our reproduction, LogReg) vs the 4 head variant
s (2 x 2 ablation), 3 seeds each
RF = {'FingerMovements': 0.5500, 'Blink': 0.9956, 'UCI-HAR': 0.9013, 'Epi
lepsy-EEG': 0.9558}
CONFIGS = [('fixed', 'concat'), ('learned', 'concat'), ('fixed', 'mixe
r'), ('learned', 'mixer')]
rows = []

def run(name, Xtr, ytr, Xte, yte):
    le = LabelEncoder().fit(ytr)
    ytr, yte = le.transform(ytr), le.transform(yte)
    t0 = time.time()
    Ttr, Tte = tokens(Xtr), tokens(Xte)
    print(f'{name}: tokens {tuple(Ttr.shape)} in {time.time() - t0:.0f}
s', flush=True)
    C, K = Ttr.shape[1], len(le.classes_)
    rows.append(dict(dataset=name, method='RF (paper protocol, reproduce
d)', mean=RF[name]))
    acc = logreg(fixed_features(Ttr), ytr, fixed_features(Tte), yte)
    rows.append(dict(dataset=name, method='LogReg (paper Sec 5.3)', mean=
acc, std=0.0))
    print(f'  LogReg {acc:.4f}', flush=True)
    for pool, chan in CONFIGS:
        t0 = time.time()
        res = [train_head(Ttr, ytr, Tte, yte, C, K, pool, chan, s) for s
in (0, 1, 2)]
        accs = [r[0] for r in res]
        rows.append(dict(dataset=name, method=f'head pool={pool} chan={ch
an}', mean=np.mean(accs), std=np.std(accs),
                         per_seed=json.dumps(accs), epochs=json.dumps([r
[1] for r in res]), params=res[0][2],
                         time_s=round(time.time() - t0)))
        print(f'  {pool:7s} {chan:6s} {np.mean(accs):.4f} +/- {np.std(acc
s):.4f} {accs} '
              f'params {res[0][2]} {time.time() - t0:.0f}s', flush=True)
    pd.DataFrame(rows).to_csv(f'{OUT}/improvement_results.csv', index=Fal
se)

def safe(fn):
    try:
```

```
        fn()
    except Exception as e:
        print('FAILED:', type(e).__name__, e, flush=True)

for name in ['FingerMovements', 'Blink']:
    safe(lambda: run(name, *load_ts_archive(name)['train'], *load_ts_arch
ive(name)['test']))
for folder, name in [('HAR', 'UCI-HAR'), ('Epilepsy', 'Epilepsy-EEG')]:
    safe(lambda: run(name, *load_tfc(folder)['train'], *load_tfc(folder)
['test']))
print(pd.DataFrame(rows)[['dataset', 'method', 'mean', 'std']].round(4).t
o_string(index=False))
```

```
FingerMovements: tokens (316, 28, 6, 33, 256) in 4s
  LogReg 0.6400
  fixed   concat 0.5833 +/- 0.0330 [0.62, 0.54, 0.59] params 73858 165
s
  learned concat 0.5333 +/- 0.0330 [0.51, 0.51, 0.58] params 141960 10
9s
  fixed   mixer  0.4867 +/- 0.0205 [0.46, 0.49, 0.51] params 203394 13
6s
  learned mixer  0.5267 +/- 0.0047 [0.52, 0.53, 0.53] params 271496 21
8s
Blink: tokens (500, 4, 6, 33, 256) in 1s
  LogReg 0.9733
  fixed   concat 0.9881 +/- 0.0042 [0.9911111111111112, 0.991111111111
1112, 0.9822222222222222] params 67714 38s
  learned concat 0.9570 +/- 0.0069 [0.9666666666666667, 0.953333333333
3334, 0.9511111111111111] params 135816 44s
  fixed   mixer  0.9489 +/- 0.0417 [1.0, 0.8977777777777778, 0.9488888
888888889] params 200322 43s
  learned mixer  0.9156 +/- 0.0031 [0.9177777777777778, 0.911111111111
1111, 0.9177777777777778] params 268424 48s
UCI-HAR: tokens (5881, 3, 6, 33, 256) in 10s
  LogReg 0.9145
  fixed   concat 0.9134 +/- 0.0060 [0.9195792331184255, 0.915167967424
4995, 0.9053274516457415] params 68998 242s
  learned concat 0.9240 +/- 0.0005 [0.9236511706820495, 0.923651170682
0495, 0.9246691550729556] params 137100 282s
  fixed   mixer  0.9025 +/- 0.0087 [0.9138106549032915, 0.892772310824
5673, 0.9009161859518154] params 200710 318s
  learned mixer  0.8898 +/- 0.0096 [0.8788598574821853, 0.902273498473
0234, 0.8883610451306413] params 268812 389s
Epilepsy-EEG: tokens (60, 1, 6, 33, 256) in 4s
  LogReg 0.9311
  fixed   concat 0.9581 +/- 0.0014 [0.9587565674255691, 0.959369527145
3591, 0.9562171628721541] params 66946 3s
  learned concat 0.9565 +/- 0.0032 [0.9528896672504378, 0.960595446584
9387, 0.9560420315236428] params 135048 3s
  fixed   mixer  0.9415 +/- 0.0224 [0.9549912434325745, 0.959632224168
1261, 0.9098949211908932] params 199938 3s
  learned mixer  0.9532 +/- 0.0064 [0.9441330998248687, 0.956742556917
6883, 0.9585814360770578] params 268040 4s
```

dataset	method	mean	std
FingerMovements	RF (paper protocol, reproduced)	0.5500	NaN
FingerMovements	LogReg (paper Sec 5.3)	0.6400	0.0000
FingerMovements	head pool=fixed chan=concat	0.5833	0.0330
FingerMovements	head pool=learned chan=concat	0.5333	0.0330
FingerMovements	head pool=fixed chan=mixer	0.4867	0.0205
FingerMovements	head pool=learned chan=mixer	0.5267	0.0047
Blink	RF (paper protocol, reproduced)	0.9956	NaN
Blink	LogReg (paper Sec 5.3)	0.9733	0.0000
Blink	head pool=fixed chan=concat	0.9881	0.0042
Blink	head pool=learned chan=concat	0.9570	0.0069
Blink	head pool=fixed chan=mixer	0.9489	0.0417
Blink	head pool=learned chan=mixer	0.9156	0.0031
UCI-HAR	RF (paper protocol, reproduced)	0.9013	NaN
UCI-HAR	LogReg (paper Sec 5.3)	0.9145	0.0000
UCI-HAR	head pool=fixed chan=concat	0.9134	0.0060
UCI-HAR	head pool=learned chan=concat	0.9240	0.0005
UCI-HAR	head pool=fixed chan=mixer	0.9025	0.0087
UCI-HAR	head pool=learned chan=mixer	0.8898	0.0096
Epilepsy-EEG	RF (paper protocol, reproduced)	0.9558	NaN
Epilepsy-EEG	LogReg (paper Sec 5.3)	0.9311	0.0000
Epilepsy-EEG	head pool=fixed chan=concat	0.9581	0.0014
Epilepsy-EEG	head pool=learned chan=concat	0.9565	0.0032
Epilepsy-EEG	head pool=fixed chan=mixer	0.9415	0.0224
Epilepsy-EEG	head pool=learned chan=mixer	0.9532	0.0064